SECURITY ASSESSMENT

Security Assessment Report

Audity — Self-Hosted Audit-Management Platform

Document ID	AUDITY-SEC-2026-06-25 / Sec2
Version	1.0 — Final
Assessment date	25 June 2026
Target version	Audity 0.2.3 (commit 8b69555 + in-progress setup-token work)
Assessment type	Authorized white-box review — static code, Docker/config, dependency, secrets & safe local runtime testing
Scope	Local source tree & local Docker environment only; no third-party systems
Prepared by	Application Security Engineer / Secure Code & Docker Reviewer
Distribution	Application owner (restricted)
Overall residual risk	**MEDIUM** — 0 Critical / 1 High / 3 Medium / 3 Low / 5 Informational
Classification	CONFIDENTIAL

This document contains the results of an authorized security assessment performed at the request of the application owner. Testing was limited to the local application and local Docker environment; no third-party systems were tested, no denial-of-service or destructive testing was performed, and no secrets were exfiltrated. Secret values are redacted throughout. Handle as CONFIDENTIAL.

Audity — Application & Docker Security Assessment

A. Executive Summary

Audity is a **well-engineered, security-conscious application**. The codebase shows deliberate, defence-in-depth security design: Argon2id password hashing with an explicit anti-timing equaliser, hashed and rotated refresh tokens in `SameSite=Strict` cookies, a double-submit CSRF token combined with an Origin allowlist, parameterised SQL throughout, AES-256-GCM for sensitive payloads, a strict Content-Security-Policy, an allowlist-only (no-shell) maintenance console gated by step-up auth, an SSRF guard on outbound connector calls, fail-closed validation that refuses to boot production with default/weak secrets, read-only non-root containers, and digest-pinned third-party images. This is materially above the security baseline of a typical application of this size.

The findings below are therefore mostly **hardening and consistency gaps**, not gaping holes. The single most important issue is **deployment-level, not code-level**: the instance is configured to be served over **plaintext HTTP on a public IP address** (`http://150.230.20.17`), which disables the cookie `Secure` flag and exposes credentials and session tokens in transit. This is a known item the owner had already chosen to defer.

Item	Rating
Overall residual risk	**Medium** (would drop to **Low** with TLS in front and the three Medium items closed)
Critical findings	0
High findings	1 (deployment/TLS — known & deferred)
Medium findings	3
Low findings	3
Informational	5

What needs urgent attention

1. **Terminate TLS in front of the app** and set `AUDITY_PUBLIC_URL` to the `https://` origin (F-01). Everything else is secondary to this on a public IP.
2. **Apply the existing SSRF guard to the webhook test endpoint** (F-02) — the protection already exists for connectors; it is simply not wired into one route.
3. **Make `AUDITY_REQUIRE_MFA` actually enforce MFA, or remove it** (F-03) — today it is a documented setting that does nothing, which is worse than no setting.

What is good: authentication/session design, CSRF/Origin handling, SQL safety, the maintenance-console architecture, secret-handling discipline, and the base container hardening are all strong. See **Section G**.

Business impact in plain English: Audity handles audit findings, evidence files, and customer sign-offs — sensitive, reputation-bearing data. The code protects that data well. The exposure that matters today is the network layer: on a public IP over HTTP, anyone able to observe the traffic (shared network, upstream hop, malicious Wi-Fi) can read login credentials and hijack sessions. Put HTTPS in front and the platform's real-world risk drops to Low.

B. Scope

In scope (assessed):

- The local source tree at `audity/` (apps `api`, `web`, `worker`, `updater`, `console-runner`; package `shared`).
- Docker build and runtime configuration: `docker-compose.yml`, `docker-compose.prod.yml`, `docker-compose.dev.yml`, all `Dockerfile`s, entrypoints, the nginx config, and install/update scripts.
- The **locally running** containers (`audity-web`, `audity-api`, `audity-worker`, `audity-updater`, `audity-db`, `audity-redis`, `audity-storage`), inspected read-only.
- Dependency manifests, secret-handling, and Git hygiene.

Out of scope / not assessed:

- Any third-party or external system. No public IP, cloud account, SaaS connector target (Jira, ServiceNow, Microsoft Graph, Power BI, etc.), SMTP server, or LLM endpoint was contacted or tested.
- Live exploitation of authenticated admin functionality (e.g., the webhook SSRF was validated by **code review only**, not by firing requests at internal infrastructure, to avoid intrusive testing).
- Networked dependency-CVE scanning (`npm audit`, `osv-scanner`, Docker Scout) — these transmit the dependency manifest to an external advisory service; per the engagement rules they were **not** run, and are recommended as a CI step instead.

- The application's business-logic correctness beyond security-relevant paths.

Authorisation & locality statement: This assessment was **explicitly authorised by the application owner**, who owns the application and its local environment. All testing was performed **locally** against `localhost` /loopback-bound containers and the local source tree. **No third-party systems were tested**, no destructive or denial-of-service testing was performed, no credentials were brute-forced, and no real secrets were exfiltrated or printed.

C. Methodology

- **Static code review** (manual) of authentication, session, CSRF, RBAC, file upload, storage, SSRF surfaces, the maintenance console, crypto utilities, and SQL access.
- **Docker & configuration review** of compose files, Dockerfiles, entrypoints, nginx, and install/update scripts, cross-checked against the **live running containers** via `docker inspect` / `docker top`.
- **Dependency review** (manual manifest/version inspection; no networked CVE DB query — see scope).
- **Secrets review** of the repo, tracked files, Git history, and compose defaults (values redacted).
- **Safe local runtime testing**: security-header inspection, unauthenticated-access checks, and container privilege inspection — all read-only, with harmless requests only.
- **Tooling:** `git`, `grep`, `curl`, `docker inspect` / `top` / `ps`. No cloud or online scanning service was used.

Limitations: No authenticated dynamic testing was performed (no live admin credentials were used), so authenticated findings (F-02) are evidenced by code rather than live PoC. No CVE database was queried; dependency findings are based on manual review of pinned versions.

D. Architecture Summary

Stack: TypeScript monorepo (npm workspaces). Backend: **Fastify 5** (Node 22). Frontend: **React + Vite**, served by **nginx 1.27-alpine**. Datastores: **PostgreSQL**, **Redis** (rate-limit + console grants + queues), **MinIO** (S3-compatible object storage for evidence and backups). Background processing: **BullMQ** worker (PDF rendering via Puppeteer/Chromium, backups via `pg_dump` / `pg_restore`). Auth: JWT (HS256) access tokens + opaque rotating refresh tokens; TOTP MFA (`otplib`) with recovery codes.

Services, exposure, and live posture (from `docker inspect` / `docker ps`):

Service	Image	Host exposure	Runtime user	Read-only FS	Notes
`audity-web` (nginx)	`nginx:1.27-alpine`	`127.0.0.1:80`	`101:101`	yes	Reverse-proxies `/api` to API
`audity-api` (Fastify)	built `node:22-alpine`	`127.0.0.1:3000`	**`node` (uid 1000)**	yes	Root only briefly in entrypoint, then drops via `su-exec`
`audity-worker`	built `node:22-alpine`	none (3001 internal)	`1000:1000`	yes	Puppeteer/Chromium, backups
`audity-updater`	built `node:22-alpine`	none (3099 internal)	**root**	**no**	**Mounts `/var/run/docker.sock` (rw)** — host-RCE-equivalent; `no-new-privileges` on live container
`audity-db`	`postgres@sha256:…` (pinned)	none	postgres	no	Not host-published
`audity-redis`	`redis@sha256:…` (pinned)	none	redis	no	Not host-published
`audity-storage` (MinIO)	`minio@sha256:…` (pinned)	`127.0.0.1:9000-9001`	root	no	API + console exposed on loopback

Trust boundaries:

- **Network:** All host-published ports bind to `127.0.0.1` only; PostgreSQL and Redis are **not** published to the host. In the dev compose, an internal-only (`internal: true`) network isolates egress; **this segmentation is absent in the production compose** (see F-04).
- **Privilege:** The **only** path to host-level compromise is the `audity-updater` container (Docker socket + root). It exposes an HTTP API on the internal network gated by a bearer token (`AUDITY_UPDATER_TOKEN`) and an allowlist of fixed `docker` subcommands run via `execFile` (no shell). The token is the host-RCE boundary, and production boot now refuses weak/default values for it.
- **Application:** RBAC permissions per role; the maintenance console additionally requires `Instance Admin` + password + MFA step-up + an IP-bound, TTL'd, Redis-backed grant.

Data flows: Browser → nginx (CSP, security headers, static assets) → `/api` → Fastify API → PostgreSQL / Redis / MinIO. Evidence files are streamed to MinIO and returned via short-lived (10 min) presigned URLs. Outbound integrations (connectors, LLM, SMTP, webhooks) egress from the API/worker.

External integrations (configured, not tested): Jira, ServiceNow, Confluence, SharePoint/OneDrive, Microsoft Entra ID, Microsoft Teams, Power BI (connectors); configurable LLM provider (Anthropic/OpenAI/Ollama); SMTP.

E. Findings Table

ID	Title	Severity	Component	Likelihood	Recommendation
F-01	Plaintext HTTP on a public IP; `Secure` cookie flag disabled; no HSTS	**High**	Deployment / `auth/routes.ts`	Medium	Terminate TLS in front; set `AUDITY_PUBLIC_URL=https://…`
F-02	SSRF via webhook "test" endpoint (no `assertSafeProviderUrl` guard)	**Medium**	`productivity/routes.ts`	Low–Med	Reuse the existing connector SSRF guard
F-03	`AUDITY_REQUIRE_MFA` is a no-op — defined/documented but never enforced	**Medium**	`auth/*`, config	Medium	Enforce it, or remove it and document MFA as per-user
F-04	Production compose lacks the dev compose's network segmentation & updater hardening	**Medium**	`docker-compose.prod.yml`	Low	Port `audity-internal` network + `no-new-privileges` / `read_only` to prod
F-05	No container resource limits (memory/PIDs/CPU)	Low	All compose services	Low	Add `mem_limit` / `pids_limit` (esp. worker)
F-06	Missing `cap_drop: [ALL]` / `no-new-privileges` on app & infra containers	Low	compose	Low	Drop capabilities; add `no-new-privileges`
F-07	App base images use floating tags (not digest-pinned); `npm install` not `npm ci`	Low	Dockerfiles	Low	Pin base digests; use `npm ci`
F-08	LLM provider outbound fetch has no SSRF/metadata guard	Info	`llm/provider.ts`	Low	Apply the connector guard (allow local-Ollama exception)
F-09	Evidence upload trusts client-declared MIME type (no content sniffing)	Info	`evidence/routes.ts`	Low	Verify magic bytes; keep `Content-Disposition: attachment`
F-10	nginx version disclosure (`Server: nginx/1.27.5`)	Info	nginx	Low	`server_tokens off;`
F-11	`install.sh` prints "Initial admin password:" (now empty by default)	Info	`scripts/install.sh`	n/a	Remove the stale line
F-12	No automated dependency-CVE scan in CI	Info	CI/CD	n/a	Add `npm audit` / `osv-scanner` to CI

F. Detailed Findings

F-01 — Plaintext HTTP on a public IP; `Secure` cookie disabled; no HSTS — High

Affected: Deployment configuration; `apps/api/src/auth/routes.ts:130-139` (`setRefreshCookie`); `apps/api/src/server.ts:115` (HSTS).

Description. The live instance is configured with `AUDITY_PUBLIC_URL=http://150.230.20.17` — a **public IPv4 address served over plaintext HTTP**. The refresh-cookie `Secure` attribute is set conditionally:

```
secure: config.publicUrl.startsWith("https://"),   // false for http:// → cookie sent in cleartext
```

and HSTS is likewise only enabled when the public URL is `https:` . So on the current deployment the session refresh cookie has **no `Secure` flag**, and there is no HSTS.

Evidence. Live API response header (read-only `curl`): `Content-Security-Policy: … connect-src 'self' http://150.230.20.17:9000 http://150.230.20.17 …` — confirming the configured public origin is `http://` on a routable public IP.

Why this matters. Login credentials, JWT access tokens (returned in the response body and replayed in the `Authorization` header), and the refresh cookie all traverse the network in cleartext. Anyone able to observe traffic on any hop between client and server — a co-located tenant, an upstream router, a hostile Wi-Fi network — can capture credentials and hijack sessions. This nullifies much of the otherwise-strong session design.

Attack scenario. A user logs in from a café/hotel network. An on-path attacker passively captures the POST to `/api/auth/login` (cleartext password) and/or the `audity_refresh` cookie, then replays it to mint fresh sessions.

Recommended remediation. Put the app behind a TLS terminator (nginx/Caddy/Traefik/cloud LB) with a valid certificate, redirect 80→443, and set `AUDITY_PUBLIC_URL=https://<host>`. This automatically enables the `Secure` cookie flag and HSTS in the existing code — no code change required. If a TLS-terminating proxy rewrites the scheme, ensure `X-Forwarded-Proto` reaches the app (the API already trusts loopback/private proxies via `trustProxy`).

Verification. `curl -sI https://<host>/ | grep -i strict-transport-security`; confirm the `audity_refresh` cookie carries `Secure`; confirm `http://` redirects to `https://`.

> **Note:** the owner had already identified and **deliberately deferred** this (the prior remediation commit excluded "H-1 (TLS)"). It is reported here at its true severity for completeness; it is an accepted, tracked risk.

F-02 — Server-Side Request Forgery via webhook "test" endpoint — Medium

Affected: `apps/api/src/productivity/routes.ts:617-646` (create + `:id/test`); schema at `:120-122`.

Description. Webhook subscriptions accept an arbitrary `targetUrl` validated only for URL syntax:

```
const webhookSchema = z.object({ … targetUrl: z.string().url(), … });
```

The test route then fetches it directly, with **no SSRF protection**:

```
const response = await fetch(webhook.rows[0].target_url, { method: "POST", … });
```

By contrast, the connector subsystem already implements a robust SSRF guard — `assertSafeProviderUrl()` (`connectors/routes.ts:349`) resolves the hostname and rejects private/loopback/link-local/CGNAT/multicast and cloud-metadata (`169.254.169.254`) addresses, and `providerFetch()` adds `redirect: "error"` and a timeout. **That guard is simply not applied to the webhook path.**

Why this matters. A holder of `settings.manage` can make the server issue POST requests to arbitrary internal destinations: cloud metadata (IMDS credential theft on a cloud VM), internal-only services on the Docker network (`http://audity-updater:3099`, MinIO console, etc.), or other hosts reachable from the API container. The response body is not returned to the caller, but the **status code, status text, and timing are** — enough for internal port/service enumeration and to trigger state-changing POSTs on internal endpoints (a semi-blind SSRF).

Likelihood. Requires an authenticated `settings.manage` account, so this is a privilege-amplification primitive rather than an unauthenticated hole — hence Medium, not High. The inconsistency is notable precisely because the correct mitigation already exists in the codebase.

Attack scenario. An admin account (or a session that has reached `settings.manage`) creates a webhook with `targetUrl=http://169.254.169.254/latest/meta-data/iam/security-credentials/` (or an internal service), clicks "test", and uses the returned status/latency to confirm reachability and pivot.

Recommended remediation. Call the existing guard before the fetch, and block redirects:

```
await assertSafeProviderUrl(webhook.rows[0].target_url);          // export it from connectors (or shared util)
const response = await fetch(webhook.rows[0].target_url, {
  method: "POST",
  headers: { "Content-Type": "application/json" },
  body: JSON.stringify({ event: "audity.webhook.test", sentAt: new Date().toISOString() }),
  redirect: "error",
  signal: AbortSignal.timeout(15_000)
});
```

Apply the same guard to any future event-driven webhook dispatch (the `events` column suggests one is planned). Consider a deny-by-default egress policy at the network layer as defence-in-depth.

Verification. With the guard in place, a webhook pointed at `http://169.254.169.254/` or `http://audity-db:5432/` must return a clear "disallowed/private address" error rather than a status code.

F-03 — `AUDITY_REQUIRE_MFA` is a non-functional control — Medium

Affected: `apps/api/src/auth/routes.ts` (login flow); `apps/api/src/config.ts`; `docker-compose*.yml`; `.env.example`.

Description. `AUDITY_REQUIRE_MFA` is surfaced as a first-class setting in `.env.example`, both compose files, and the documented configuration. **It is never read or enforced anywhere in the API.** A repo-wide search for the variable / any `requireMfa` logic finds only the *per-user* check in the login flow ("does this specific user have MFA enabled?"). There is no code path that forces a user without MFA to enrol, nor that rejects a password-only login when `AUDITY_REQUIRE_MFA=true`.

Evidence.

```
$ grep -rn "REQUIRE_MFA\|requireMfa" apps/api/src
apps/api/src/auth/routes.ts:317:        mfaRequired: true,   ← per-user only; AUDITY_REQUIRE_MFA never referenced
```

Why this matters. This is worse than simply leaving MFA optional. An operator who reads the configuration, sets `AUDITY_REQUIRE_MFA=true`, and reasonably believes MFA is now mandatory for all accounts is operating under a **false sense of security** — accounts can still authenticate with a password alone. For an internet-reachable instance, single-factor auth on privileged accounts is the practical attack surface for credential stuffing/phishing.

Attack scenario. Admin enables `AUDITY_REQUIRE_MFA=true`, onboards staff, and never verifies enrolment. A phished password on a non-enrolled account grants full access; the "required MFA" assumption silently never applied.

Recommended remediation. Either:

- **Enforce it:** when `AUDITY_REQUIRE_MFA=true`, block issuance of a full session for users without MFA and route them through a forced-enrolment step (and/or refuse password-only logins for privileged roles); or
- **Remove it** from config/compose/docs and document MFA as per-user opt-in (the maintenance console already *does* hard-require MFA, which is the correct pattern to extend).

A no-op security toggle should not ship.

Verification. With enforcement, set `AUDITY_REQUIRE_MFA=true`, create a user without MFA, and confirm they cannot obtain an authenticated session without enrolling.

F-04 — Production compose lacks the dev compose's segmentation & updater hardening — Medium

Affected: `docker-compose.prod.yml` (no `networks:` section; `audity-updater` service).

Description. The base/dev `docker-compose.yml` is carefully segmented:

- an `audity-internal` network with `internal: true` (no internet egress),
- the Docker-socket `audity-updater` placed on that network only, behind `profiles: ["console"]`, with `security_opt: [no-new-privileges:true]`.

The **production** compose (`docker-compose.prod.yml`) — the file intended for real deployments — has **no `networks:` section at all** (every service shares the default bridge with full internet egress), runs the Docker-socket updater **always-on** (no `profiles`), with **no `no-new-privileges`, no `read_only`, and no `cap_drop`**, and additionally bind-mounts the entire project directory read-write (`${AUDITY_HOST_PROJECT_DIR:-.}:/audity`).

Evidence.

```
$ grep -n "networks:\|internal:" docker-compose.yml        → audity-internal / internal: true present
$ grep -n "networks:\|internal:" docker-compose.prod.yml   → (no networks section in prod compose)
```

Why this matters. The host-RCE boundary in production rests **entirely** on the `AUDITY_UPDATER_TOKEN` (mitigated by the M-4 fail-closed validation — good). But with a flat network and an always-on root+socket container, a compromise of any lower-value container (e.g., the worker via a dependency or Puppeteer issue) sits on the same network as the updater and can both reach the internet freely (exfiltration) and attempt the updater's token-gated API. The dev compose's egress isolation and profile-gating materially shrink that window; production should not be *less* hardened than dev.

Recommended remediation. Port the dev hardening to `docker-compose.prod.yml`:

- recreate the `audity-internal` (`internal: true`) network and place the updater (and DB/Redis/storage) on it appropriately;
- add `security_opt: [no-new-privileges:true]`, and (where the workload allows) `read_only: true` + `cap_drop: [ALL]` to the updater;
- if the console/updater is not always needed, gate it behind a profile so the socket-holder is not continuously running.

Verification. `docker inspect audity-updater` shows `no-new-privileges`; `docker network inspect` shows the updater on an `internal` network; the updater cannot reach the internet (`docker exec audity-updater wget -T3 http://example.com` fails).

F-05 — No container resource limits — Low

Affected: all services in both compose files (`Memory=0`, `PidsLimit=<nil>` confirmed via `docker inspect`).

No `mem_limit`, `pids_limit`, or CPU constraints are set. A heavy or malicious workload — a large Puppeteer PDF render, a big framework YAML/CSV import, a `docker system prune`, or simply a memory leak — can consume unbounded host memory/PIDs and degrade or crash the host (the prior worker death the team already saw is a related availability concern). **Remediation:** set conservative `mem_limit` and `pids_limit` per service, with the most generous budget on `audity-worker`. **Verification:** `docker inspect <svc> --format '{{.HostConfig.Memory}}'` is non-zero.

F-06 — Missing `cap_drop` / `no-new-privileges` on app & infra containers — Low

Affected: `audity-web`, `audity-api`, `audity-worker`, `audity-db`, `audity-redis`, `audity-storage` (`CapDrop=[]` confirmed live).

The app containers already run **non-root + read-only**, which is the bulk of the benefit, but they retain the default Linux capability set and lack `no-new-privileges`. Dropping all capabilities (`cap_drop: [ALL]`, re-adding only what each image needs) and adding `no-new-privileges:true` is low-risk, standard hardening that further limits post-exploitation. For `audity-api`, `no-new-privileges` is compatible with the root→`node` `su-exec` drop (a privilege *drop*, not gain). **Verification:** `docker inspect` shows the dropped caps and `no-new-privileges`.

F-07 — Floating base-image tags & non-deterministic installs — Low

Affected: `apps/*/Dockerfile`.

Third-party service images (`postgres`, `redis`, `minio`) are correctly **digest-pinned**, but the application images build `FROM node:22-alpine` and `FROM nginx:1.27-alpine` — **floating tags** that can resolve to a different image on each rebuild, undermining reproducibility and supply-chain integrity. The Dockerfiles also use `RUN npm install` rather than `npm ci`, so the committed `package-lock.json` is not strictly enforced at build time. **Remediation:** pin base images by digest (`node:22-alpine@sha256:…`) and switch builds to `npm ci`. **Verification:** rebuild yields identical base layers; `npm ci` fails on lockfile drift.

F-08 — LLM provider outbound fetch has no SSRF/metadata guard — Informational

Affected: `apps/api/src/llm/provider.ts:171,236,308`.

The configurable LLM endpoint is fetched without the connector SSRF guard. This is admin-configured and *intended* to point at an arbitrary endpoint (including a local Ollama at `host.docker.internal:11434`), so it is lower risk than F-02 — but it is the same class of admin-controlled outbound request and would benefit from at least blocking cloud-metadata addresses while allowing an explicit local-Ollama exception.

F-09 — Evidence upload trusts client-declared MIME type — Informational

Affected: `apps/api/src/evidence/routes.ts:64-90`.

Upload type enforcement compares `file.mimetype` (the **client-declared** content type) against the allowlist, and stores it back as the object's `Content-Type`; there is no magic-byte/content sniffing. Residual risk is **low**: the allowlist excludes actively-dangerous types (no `text/html`, no `application/javascript`), files are served from a **separate storage origin** via short-lived presigned URLs, and `X-Content-Type-Options: nosniff` is set. A mislabeled/polyglot file could still be stored. **Hardening:** sniff magic bytes server-side and force `Content-Disposition: attachment` on evidence downloads.

F-10 — nginx version disclosure — Informational

`Server: nginx/1.27.5` is returned (confirmed live). Set `server_tokens off;` to avoid advertising the exact version.

F-11 — `install.sh` prints an empty "Initial admin password" — Informational

`scripts/install.sh:181` still echoes `Initial admin password: $(env_value AUDITY_SEED_ADMIN_PASSWORD)`. After the setup-token refactor the seed password is unset by default, so this prints an empty value and is confusing. Remove the stale line (the new one-time setup-token line at `:179` supersedes it).

F-12 — No automated dependency-CVE scanning — Informational

No `npm audit` / `osv-scanner` / Docker Scout step exists in CI. Manual review found the dependency set current (Fastify 5, `jsonwebtoken` 9, `argon2` 0.41, `@fastify/helmet` 13, `zod` 4, `ws` 8.21, `yaml` 2.9, `otplib` 12) with no obviously vulnerable pins, but a CVE-DB cross-check is the proper control. (Not run here because it ships the manifest to an external advisory service — see scope.)

G. Positive Observations

The following controls are already implemented well and should be preserved:

- **Password handling:** Argon2id everywhere, with an explicit **timing equaliser** (`auth/service.ts:115-133`) so non-existent/disabled accounts cost the same time as a real verify — defeats username enumeration via timing.
- **Session & tokens:** JWT HS256 with `issuer` + expiry validation and a fixed algorithm allowlist (no `alg` confusion); access tokens carried in the `Authorization` header (immune to CSRF); refresh tokens are 48 random bytes, **SHA-256-hashed at rest**, **rotated on every refresh**,

and set as `HttpOnly; SameSite=Strict`, path-scoped cookies; idle-timeout enforced atomically; **all sessions revoked on password change**.

- **CSRF:** double-submit CSRF token (hashed at rest) **plus** an Origin-allowlist preHandler on every state-changing method, with a same-origin fallback that still rejects genuine cross-site requests.
- **Fail-closed secrets:** `validateProductionConfig` refuses to boot production with default, weak (<32/24 byte), or duplicated (`ENCRYPTION_KEY == APP_SECRET`) secrets, including the Docker-socket updater token.
- **SQL:** parameterised queries throughout; no string interpolation into SQL observed across the reviewed routes.
- **Crypto:** AES-256-GCM with a random 12-byte IV and auth tag for sensitive payloads; SMTP passwords encrypted at rest.
- **Maintenance console:** allowlist-only, **no shell anywhere** (fixed `docker` argv via `execFile`), `Instance Admin` + password + **MFA step-up**, IP-bound TTL'd Redis grant, fully audited, **off by default**, and the privileged Docker-socket grant confined to a single token-gated service.
- **SSRF guard on connectors:** DNS-resolution-based private/metadata blocklist + `redirect:"error"` + timeout (the pattern F-02 should reuse).
- **Network posture:** DB/Redis not host-published; web/api/storage bound to loopback; egress-isolated internal network in dev.
- **Container hardening:** read-only rootfs + non-root + tmpfs scratch on app containers; entrypoint drops root via `su-exec`; third-party images digest-pinned.
- **First-run hardening (in progress):** one-time setup-token gate against first-admin hijack, with `pg_advisory_lock` serialisation of setup; the unauthenticated recovery-phrase verify oracle was removed and moved behind auth.
- **Secret hygiene:** `.env` is `0600` (verified live); **no** secrets in tracked files or Git history; strong `.gitignore` / `.dockerignore`; nginx blocks dotfiles, source dirs, config files, and source maps.
- **Auditability:** comprehensive audit logging of auth and security-relevant events; mandatory 24h log archival.

H. Quick Wins (high value, low effort)

1. **F-02:** wire the existing `assertSafeProviderUrl` + `redirect:"error"` into the webhook test fetch (≈4 lines). *(concrete patch shown in F-02; can be applied on request)*
2. **F-03:** remove the dead `AUDITY_REQUIRE_MFA` toggle (or enforce it) — eliminates a misleading control.
3. **F-10:** `server_tokens off;` in the nginx server block.
4. **F-11:** delete the stale "Initial admin password" echo in `install.sh`.
5. **F-06:** add `security_opt: [no-new-privileges:true]` to the app services.
6. **F-12:** add an `npm audit --omit=dev` (or `osv-scanner`) job to CI.

I. Hardening Recommendations

Application code

- Centralise outbound-request egress through a single guarded helper (extend the connector guard) and use it for webhooks (F-02) and the LLM provider (F-08).
- Remove or implement `AUDITY_REQUIRE_MFA` (F-03); consider extending the console's hard-MFA pattern to privileged roles.
- Add server-side content sniffing + `Content-Disposition: attachment` for evidence (F-09).

Dockerfile

- Pin base images by digest; switch to `npm ci`; keep multi-stage + non-root + read-only (F-07).

Docker Compose

- Bring `docker-compose.prod.yml` up to the dev compose's segmentation (`internal` network) and add `no-new-privileges` / `cap_drop` / `read_only` + resource limits across services; gate the socket-holding updater behind a profile (F-04, F-05, F-06).

Secrets management

- Continue the `.env` (0600) + fail-closed model. Consider Docker secrets / a secrets manager for the updater and DB credentials so they are not present in every container's environment.

Authentication / session handling

- Deploy behind HTTPS to activate `Secure`/HSTS (F-01). Optionally add refresh-token **reuse detection** (revoke the whole session family if a rotated token is replayed).

Authorization

- The RBAC + per-route `requirePermission` / `requireCsrfPermission` model is sound; keep enforcing object-level access checks (`canAccessAssessment`) as done in `evidence/routes.ts`.

Database / storage

- Keep DB/Redis off the host; ensure MinIO presigned-URL TTLs stay short (currently 10 min — good). Consider a non-root MinIO runtime where supported.

Network exposure

- Maintain loopback-only host binds; add an egress-deny default on the internal network in production.

Logging / monitoring

- Maintain the redaction in the request logger (`redactSensitiveQuery`); add alerting on `auth.login.failed` spikes and `console.*` events.

CI/CD & dependency management

- Add dependency-CVE scanning, image scanning (Trivy/Grype/Scout), and a secrets scanner (gitleaks) to CI; enforce `npm ci` reproducibility.

Developer workflow

- Keep `AUDITY_ALLOW_INSECURE_DEFAULTS` strictly dev-only; never set it in any production-adjacent environment.

J. Commands Run (read-only; secrets redacted)

Command (summarised)	Purpose	Result
`git log/diff` , `git ls-files`	Repo & change review, tracked-secret check	No secrets tracked or in history; reviewed in-progress setup-token work
`grep -rn` for `child_process` , `eval` , `yaml` , `fetch` , SQL patterns, `REQUIRE_MFA` , SSRF guard	Dangerous-sink & control discovery	No `eval` ; no shell-based command construction in app paths; SSRF guard present on connectors but not webhooks; `AUDITY_REQUIRE_MFA` unused
`curl -sS -D-` on `:3000/health` , `/api/auth/me` , `/api/auth/setup-status` , `:80/`	Security headers & auth gating	Strong CSP/headers; 401 on protected route; **public-IP `http://` origin observed**
`docker ps` , `docker inspect` , `docker top`	Live container posture	DB/Redis not host-published; api runs as uid 1000; updater root+socket; no caps dropped; no mem limits
`command -v` for scanners / PDF tools	Tooling inventory	No networked scanners run (per scope); Chrome used for offline PDF
`ls -la .env`	Secret-file permission check	`.env` is `0600` (verified)

No command printed real secret values; compose **defaults** referenced are placeholders (`change-me` , `replace-me` , `replace-with-base64-encoded-32-byte-key`) and are redacted as such.

K. Risk-Ranked Remediation Roadmap

Fix immediately

- F-01: Put TLS in front; set `AUDITY_PUBLIC_URL=https://…` (activates `Secure` cookie + HSTS).
- F-02: Apply the SSRF guard to the webhook test endpoint.

Fix this week

- F-03: Enforce or remove `AUDITY_REQUIRE_MFA` .
- F-04: Align `docker-compose.prod.yml` hardening with the dev compose.
- F-10/F-11: `server_tokens off;` ; remove the stale install.sh echo.

Fix this month

- F-05/F-06: Add resource limits, `cap_drop: [ALL]` , `no-new-privileges` across services.
- F-07: Digest-pin base images; `npm ci` .
- F-12: Add dependency/image/secret scanning to CI.

Longer-term

- F-08/F-09: Centralised guarded egress; evidence content-sniffing.
- Refresh-token reuse detection; Docker secrets for credential distribution; egress-deny default on the internal network.

L. Appendix

Tooling summary. Manual static review + `git` / `grep` / `curl` / `docker` inspection. No cloud or online scanning service was used. PDF generated locally via headless Google Chrome (`--print-to-pdf`); no content was uploaded anywhere.

Assumptions.

- The live containers correspond to the reviewed source (commit `8b69555` + the uncommitted setup-token work).
- The public IP in `AUDITY_PUBLIC_URL` reflects the intended deployment origin; host port binds remain loopback-only, implying an external path (proxy/forwarding) provides reachability.
- "Production" deployments use `docker-compose.prod.yml` .

Limitations / tests intentionally not performed (for safety or scope).

- No authenticated dynamic exploitation (no live admin credentials used); F-02 validated by code only.
- No denial-of-service, brute-force, fuzzing, or persistence testing.
- No third-party/connector/SMTP/LLM endpoints contacted.
- No networked CVE database queried (manifest not shipped to an external advisory service).
- Git history scanned for filenames/patterns, not exhaustively content-diffed across all branches.

Dependency snapshot (API, manual review — not a CVE verdict): `fastify ^5.8.5` , `@fastify/helmet ^13` , `@fastify/rate-limit ^10.3` , `@fastify/cors ^11.2` , `argon2 ^0.41.1` , `jsonwebtoken ^9.0.2` , `otplib ^12.0.1` , `pg ^8.12` , `ioredis ^5.11` , `minio ^8.0.7` , `nodemailer ^9.0.1` , `bullmq ^5.78` , `yaml ^2.9.0` , `ws ^8.21` , `zod ^4.4.3` .

End of report. Prepared for the application owner under explicit local-testing authorization. No third-party systems were tested; no secrets were exfiltrated.